SCOUT - Cambios exactos para Early Burst (EB)

Comparación del árbol de trabajo actual con el commit original HEAD: c72befe0d052f3960c448a64bff776665ab8ea4c

Las secciones siguientes enumeran cada archivo modificado y reproducen el diff unificado completo: líneas con + añadidas, con - retiradas y sin prefijo como contexto. Los marcadores " □" solo indican una continuación visual de una línea larga en el PDF; no son parte del código. No se incluyen los PDF binarios en el diff de código.

Resumen funcional

R/SCOUT_EM_utils.R: Nuevas compute_VCV_EB(), transformPhy_EB() y compute_W_EB(); EB como modelo de un régimen en makeEdges(), formatSCOUT() y preprocessTree(); inicialización en preprocessGene(); extracción de theta, soporte de lotes y matriz de combinaciones para un solo gen.

R/SCOUT_EM.R: Despacho EB en e_step(); nueva m_step_EB() con beta no positivo y sus cotas basadas en H; integración en m_step_tipfog() y run_em(); beta y beta_H en historial. La ruta de ajuste BM/OU permanece independiente.

R/analysis_utils.R: extract_parameters() elimina el régimen artificial de raíz también para EB; annotate_history() reconoce EB en los nombres de genes. El recuento de parámetros mantiene la excepción únicamente para BM1.

R/model_selection.R: hybrid_decide() trata EB como candidato simple evaluado por AIC/AICc, no como alternativa OU multirrégimen sujeta a LRT; scout_model_selection() incluye beta, beta_H y el indicador beta_near_bm.

R/dropout_validation.R: fit_paras(), predict_masked_latent() y runSCOUT.dropout() permiten EB; EB no se confunde con una columna de estados; se guardan beta y beta_H en resultados de validación.

R/main.R: SCOUT() documenta EB y añade beta y beta_H a la salida de mejores ajustes.

README.md: Descripción pública de EB, beta <= 0, raíz fija, escala de ramas, beta_H y el indicador cerca de BM1.

man/SCOUT.Rd: Ayuda de SCOUT() actualizada para EB.

tests/testthat/test-early-burst.R: Pruebas nuevas: covarianza densa frente a árbol transformado, límite beta=0, rechazo de beta positivo, raíz fija, reescalado de ramas, EM, predicción, AIC y selección híbrida; ajustes de uno y varios genes.

Parámetros y criterio usados en EB

beta <= 0. H = altura máxima del árbol empleado. Inicialización beta = -1/H; cota inferior log(1e-5)/H y superior 0. sigma es tasa de varianza. Covarianza sigma * expm1(beta * s) / beta, con límite estable cerca de beta*H=0. El indicador beta_near_bm equivale a abs(beta_H) <= 0.01 y no altera la selección por AIC/AICc. Para EB se condiciona en el valor ancestral estimado, sin varianza aleatoria adicional en la raíz.

Diff íntegro por archivo (incluye funciones y líneas modificadas)

R/SCOUT_EM_utils.R

Total: 188 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/SCOUT_EM_utils.R b/R/SCOUT_EM_utils.R
index 8d1af55..b044a62 100644
--- a/R/SCOUT_EM_utils.R
+++ b/R/SCOUT_EM_utils.R
@@ -386,6 +386,46 @@ compute_VCV<- function(tree_info, alpha, sigma, add.root=TRUE) {
   return(V)
 }

+# EARLY BURST (EB): beta is per unit of branch length. beta * H is
+# dimensionless, so the small-beta limit is stable on trees of any height.
+compute_VCV_EB <- function(tree_info, beta, sigma) {
+  s <- tree_info$shared_lengths
+  H <- max(tree_info$leaf_dists)
+  if (!is.finite(beta) || beta > 0 || !is.finite(sigma) || sigma <= 0 ||
+      !is.finite(H) || H <= 0) stop('EB requires beta <= 0, sigma > 0 and a positive tree height.')
+  V <- if (abs(beta) * H < 1e-8) {
+    # Include the first correction rather than abruptly replacing EB by BM.
+    sigma * s * (1 + beta * s / 2)
+  } else {
+    sigma * expm1(beta * s) / beta
```

```
+  }
+  dimnames(V) <- NULL
+  V
+}
+
+# EB's three-point covariance is the BM covariance of this transformed tree.
+# The root has an estimated mean but no extra random-root variance.
+transformPhy_EB <- function(phy, beta, sigma) {
+  phy <- ape::reorder.phylo(phy, 'cladewise')
+  depths <- ape::node.depth.edgelength(phy)
+  a <- depths[phy$edge[, 1]]
+  dt <- phy$edge.length
+  H <- max(depths[seq_along(phy$tip.label)])
+  if (!is.finite(beta) || beta > 0 || !is.finite(sigma) || sigma <= 0 ||
+      !is.finite(H) || H <= 0) stop('EB requires beta <= 0, sigma > 0 and a positive tree height.')
+  phy$edge.length <- if (abs(beta) * H < 1e-8) {
+    sigma * exp(beta * a) * dt * (1 + beta * dt / 2)
+  } else {
+    sigma * exp(beta * a) * expm1(beta * dt) / beta
+  }
+  phy$root.edge <- 0
+  list(tree = phy, diagWeight = stats::setNames(rep(1, length(phy$tip.label)), phy$tip.label))
+}
+
+compute_W_EB <- function(tree_info) {
+  matrix(1, nrow = tree_info$n_leaves, ncol = 1L)
+}
+
 build_regime_paths <- function(tree, node_regimes, root_dists) {
   n_tips <- length(tree$tip.label)
   root <- n_tips + 1
@@ -610,7 +650,7 @@ makeEdges <- function(phy, data, model, k, Tmax_init, int.state, root.age, scale

     edges=edges[sort.list(edges[,3]),]

-    if (model %in% c('BM1', 'OU1')) {
+    if (model %in% c('BM1', 'OU1', 'EB')) {
         regime <- matrix(0,nrow=length(edges[,1]),ncol=k)
         regime[,1] <- 1
         if (k>1) {
@@ -756,7 +796,10 @@ preprocessTree <- function(inputs, reg,
     data.orig <- inputs$meta_data[, c('species', reg_col, gene_cols)]
     phy <- reorder.phylo(phy, "cladewise")

-    if(model == "BM1" & root.fixed == FALSE) { root.fixed = TRUE } # change the argument if BM1.
+    if(model %in% c('BM1', 'EB') & root.fixed == FALSE) { root.fixed = TRUE } # BM and EB have no stationar
+    y root.
+    if (model == 'EB' && (!(algorithm %in% c('three.point', 'inverse')) || weight_type != 'alt')) {
+        stop('EB requires weight_type = "alt" and algorithm = "three.point" or "inverse".')
+    }

     data <- data.frame(data.orig[, c(reg_col, gene_cols)], row.names = data.orig[, 'species'])

@@ -767,7 +810,7 @@ preprocessTree <- function(inputs, reg,
     Tmax.i <- max(MakeAgeTable(phy, root.age=root.age)) # with root.age == NULL

         #### Establish a bunch of values used later.
-    if (model %in% c('BM1', 'OU1')){
+    if (model %in% c('BM1', 'OU1', 'EB')){
           #Begins the construction of the edges matrix -- similar to the ouch format##
           #Makes a vector of absolute times in proportion of the total length of the tree
           k <- 1
@@ -881,7 +924,7 @@ preprocessGene <- function(inputs, gene_name, tree_const, root.state, root.fixed

         # build theta0 into theta if asking to get root theta.

-        if (model == 'BM1'){
+        if (model %in% c('BM1', 'EB')){
             theta <- c( means.by.reg.init)
           #  names(theta) <- c('root')
         } else {
@@ -902,7 +945,11 @@ preprocessGene <- function(inputs, gene_name, tree_const, root.state, root.fixed
         tau.i <- 0
     }

-    par0 <- list('tau' = tau.i, 'alpha' = alpha.i, 'sigma' = sigma.i, 'theta' = theta) # tau, alpha, sigma2
-    , means.by.regime.
+    if (model == 'EB') {
+        par0 <- list(tau = tau.i, beta = -1/Tmax, sigma = sigma.i, theta = theta)
+    } else {
+        par0 <- list('tau' = tau.i, 'alpha' = alpha.i, 'sigma' = sigma.i, 'theta' = theta)
```

```
+    }

     return(list(param_init = par0, X = X))

@@ -974,7 +1021,7 @@ formatSCOUT <- function(tree_path, metadata_path, outpath,
         }
     }

-    reg_oums <- regimes[! regimes %in% c('BM1', 'OU1')]
+    reg_oums <- regimes[! regimes %in% c('BM1', 'OU1', 'EB')]
     if (length(intersect(reg_oums, colnames(meta))) != length(reg_oums)){
         stop(sprintf('Specified regimes not found in metadata columns: %s. Please correct. Exiting...', pas
     te(reg_oums, collapse=', ')))
     }
@@ -1090,7 +1137,7 @@ formatSCOUT <- function(tree_path, metadata_path, outpath,
     # Kind of convoluted but if there are multiple OUM regimes, need to store a tree for each potential reg
     ime.
     for (r in 1:length(regimes)){
         rname <- regimes[r]
-        if (length(unique(cln_meta[[rname]])) > 1) {
+        if (!rname %in% c('BM1', 'OU1', 'EB') && length(unique(cln_meta[[rname]])) > 1) {
             model <- 'OUM'
             if (anc_infer == 'skip'){
                 phy$node.label <- rep('ANC', phy$Nnode)
@@ -1150,8 +1197,7 @@ runSCOUT<- function(idata,

     genes <- idata$gene_cols
     regimes <- idata$regimes
-    combos <- expand.grid(genes, regimes)
-    combos <- apply(combos, 2, as.character)
+    combos <- as.matrix(expand.grid(genes, regimes, stringsAsFactors = FALSE))
     colnames(combos) <- c('gene_name', 'regime')#, 'burnin', 'lambda1', 'tau_prior_mean', 'tau_prior_sd', '
     lambda2')
     code <- if (is.null(runid)) {paste(sample(c(0:9, letters, LETTERS), 8, replace = TRUE), collapse = "")
     } else {runid}

@@ -1248,8 +1294,7 @@ runSCOUT.batches <- function(idata,
     }

     regimes <- idata$regimes
-    combos <- expand.grid(genes, regimes)
-    combos <- apply(combos, 2, as.character)
+    combos <- as.matrix(expand.grid(genes, regimes, stringsAsFactors = FALSE))
     colnames(combos) <- c('gene_name', 'regime')#, 'burnin', 'lambda1', 'tau_prior_mean', 'tau_prior_sd', '
     lambda2')
     code <- if (is.null(runid)) {paste(sample(c(0:9, letters, LETTERS), 8, replace = TRUE), collapse = "")
     } else {runid}

@@ -1312,7 +1357,7 @@ runSCOUT.batches <- function(idata,
           }

           # Run EM
-          res1 <- run_em(tree_constants$tree, gene_info$X, gene_info$param_init,
+           res1 <- run_em(tree_constants$tree, gene_info$X, gene_info$param_init,
                          tree_constants$edges, defaults,
                          max_iter = 100,
                          tol_rel = 1e-4,
@@ -1325,7 +1370,7 @@ runSCOUT.batches <- function(idata,
                          save = FALSE,
                          save_pre = NULL,
                          skipE = M_only,
-                         verbose = verbose)
+                          verbose = verbose)

           res1$settings <- as.vector(combos[j, ])
           batch_results[[length(batch_results) + 1]] <- res1
@@ -1374,6 +1419,8 @@ combine_disk_batch <- function(results_dir, tid, batch_size=10, tmpdir='./tmp',
           regime = sapply(batch_results, function(x) x$settings[2]),
           loglik = sapply(batch_results, function(x) x$final$ll_total),
           alpha = sapply(batch_results, function(x) x$final$alpha),
+          beta = sapply(batch_results, function(x) x$final$beta),
+          beta_H = sapply(batch_results, function(x) x$final$beta_H),
           sigma = sapply(batch_results, function(x) x$final$sigma),
           objective = sapply(batch_results, function(x) x$final$objective),
           tau = sapply(batch_results, function(x) x$final$tau),
@@ -1400,8 +1447,8 @@ extract_parameters <- function(results_list){
         reg <- res$settings[[2]]
         unq_regs <- res$defaults$parsed_alt_tree$unique_regimes

-        if (reg == 'BM1'){
-            unq_regs <- unq_regs[-1] # strip off root if BM1.
```

```
+        if (reg %in% c('BM1', 'EB')){
+            unq_regs <- unq_regs[-1] # BM1 and EB share a fixed-root, single-theta mean.
         }
         tt <- unlist(res$paras$theta)
         names(tt) <- paste0('theta_', unq_regs)
@@ -1493,7 +1540,7 @@ verify_initial_params <- function(paras, defaults, verbose=TRUE) {
     paras$sigma <- 0.1
   }

-  if (paras$alpha < 1e-6 && defaults$model != 'BM1') {
+  if (defaults$model != 'EB' && defaults$model != 'BM1' && paras$alpha < 1e-6) {
     if (verbose) cat("WARNING: Alpha suspiciously small, resetting...\n")
     paras$alpha <- 0.1
   }
```

R/SCOUT_EM.R

Total: 148 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/SCOUT_EM.R b/R/SCOUT_EM.R
index dd1b3cc..e744185 100644
--- a/R/SCOUT_EM.R
+++ b/R/SCOUT_EM.R
@@ -4,7 +4,8 @@
 e_step <- function(X, phy, edges, paras, defaults, diagnose=TRUE){
     #V <- quickVCV(phy, paras$alpha, paras$sigma, defaults$scaleHeight)
     if (defaults$weight_type == 'alt'){
-      V <- compute_VCV(defaults$parsed_alt_tree, paras$alpha, paras$sigma, add.root=defaults$add.root) # st
     op gap will fix.
+      V <- if (defaults$model == 'EB') compute_VCV_EB(defaults$parsed_alt_tree, paras$beta, paras$sigma) el
     se
+        compute_VCV(defaults$parsed_alt_tree, paras$alpha, paras$sigma, add.root=defaults$add.root)
     } else {
       V <- varcov.ou(phy, edges, paras$alpha, paras$sigma, defaults$root.state, defaults$scaleHeight, defau
     lts$assume.station)
     }
@@ -59,7 +60,8 @@ e_step <- function(X, phy, edges, paras, defaults, diagnose=TRUE){
     }

     if (defaults$weight_type == 'alt'){
-        W_ <- compute_W_matrix(defaults$parsed_alt_tree, paras$alpha, add.root=defaults$add.root )
+        W_ <- if (defaults$model == 'EB') compute_W_EB(defaults$parsed_alt_tree) else
+          compute_W_matrix(defaults$parsed_alt_tree, paras$alpha, add.root=defaults$add.root )
     } else {
       W_ <- weight.mat(phy, edges, paras$alpha, defaults$root.state, assume.station=defaults$assume.station
     )
     }
@@ -115,9 +117,70 @@ loglik_OU_inverse <- function(X, W, V){
   return(logl)
 }

+# EARLY BURST (EB): beta is signed, unlike OU alpha and the positive parameters.
+# Keep this optimizer separate so BM1/OU parameterization and priors are untouched.
+m_step_EB <- function(Z, X, params_init, phy, defaults, lambda1) {
+  H <- max(defaults$parsed_alt_tree$leaf_dists)
+  # mvMORPH's default minimum rate is 1e-5 at the tree height.
+  beta_min <- log(1e-5) / H
+  has_tau <- !defaults$skipTau
+  lower <- c(beta_min, log(0.01), log(1e-10), if (has_tau) log(0.05))
+  upper <- c(0, log(100), log(1000), if (has_tau) log(2))
+  x0 <- c(params_init$beta, log(params_init$sigma), log(params_init$theta),
+          if (has_tau) log(params_init$tau))
+  x0 <- check_bounds(x0, lower, upper, verbose = defaults$verbose)
+  W <- compute_W_EB(defaults$parsed_alt_tree)
+  expected <- function(theta) {
+    mu <- W %*% theta
+    rownames(mu) <- phy$tip.label
+    mu
+  }
+  loglik <- function(par) {
+    beta <- par[1]
+    sigma <- exp(par[2])
+    theta <- exp(par[3])
+    tau <- if (has_tau) exp(par[4]) else 0
+    if (defaults$algorithm == 'three.point') {
+      tr <- transformPhy_EB(phy, beta, sigma)
+      if (has_tau) {
+        tips <- tr$tree$edge[, 2] <= length(phy$tip.label)
+        tr$tree$edge.length[tips] <- tr$tree$edge.length[tips] + tau^2
+      }
```

```
+      ll <- loglik_OU_threepoint(Z, list(tr, expected(theta), length(Z)))
+    } else {
+      V <- compute_VCV_EB(defaults$parsed_alt_tree, beta, sigma)
+      if (has_tau) diag(V) <- diag(V) + tau^2
+      ll <- loglik_OU_inverse(Z, W, V)
+    }
+    list(ll = as.numeric(ll), tau = tau)
+  }
+  objective <- function(par) {
+    val <- loglik(par)
+    # EB has no OU alpha/Gamma prior; retain the existing tau regularization.
+    -(val$ll + length(X) * lambda1 *
+      dnorm(val$tau, mean = sd(X) * defaults$tau_prior_mean,
+            sd = defaults$tau_prior_sd, log = TRUE))
+  }
+  fit <- tryCatch(nloptr(x0 = x0, eval_f = objective,
+                         opts = list(algorithm = 'NLOPT_LN_SBPLX', maxeval = 200,
+                                     ftol_rel = 1e-8, ftol_abs = 1e-10),
+                         lb = lower, ub = upper), error = function(e) NULL)
+  if (is.null(fit)) {
+    if (defaults$verbose) warning('EB optimization failed; retaining initial parameters.')
+    sol <- x0
+  } else {
+    sol <- fit$solution
+  }
+  val <- loglik(sol)
+  list(beta = sol[1], sigma = exp(sol[2]), theta = exp(sol[3]),
+       tau = val$tau, ll_ou = val$ll, ll_total = val$ll,
+       objective = -objective(sol))
+}
+
 #### DRAFT of Joint M-step! ####
 # M-step for (phi, alpha, sigma, theta)
 m_step_tipfog <- function(Z, X, params_init, phy, edges, defaults, lambda1, lambda2) {
+  if (defaults$model == 'EB') return(m_step_EB(Z, X, params_init, phy, defaults, lambda1))

   objective <- function(params_latent) {
       par <- exp(params_latent)
@@ -419,6 +482,8 @@ run_em <- function(phy, X, params_init, edges, defaults,
     ll_noise = numeric(),
     sigma = numeric(),
     alpha = numeric(),
+    beta = numeric(),
+    beta_H = numeric(),
     tau = numeric()
   )
   monitor_history <- NULL
@@ -449,6 +514,8 @@ run_em <- function(phy, X, params_init, edges, defaults,

     if (defaults$model == 'BM1'){
         pset <- c("sigma", "theta", "tau")
+    } else if (defaults$model == 'EB') {
+        pset <- c('beta', 'sigma', 'theta', 'tau')
     } else {
         pset <- c("alpha", "sigma", "theta", "tau")
     }
@@ -458,7 +525,11 @@ run_em <- function(phy, X, params_init, edges, defaults,
     }

     m_step1_result <- m_step_tipfog(e_result$E_Z, X, params[pset], phy, edges, defaults, lambda1, lambda2)
-    params$alpha <- m_step1_result$alpha
+    if (defaults$model == 'EB') {
+      params$beta <- m_step1_result$beta
+    } else {
+      params$alpha <- m_step1_result$alpha
+    }
     params$sigma <- m_step1_result$sigma
     params$theta <- m_step1_result$theta
     params$tau <- m_step1_result$tau
@@ -480,13 +551,19 @@ run_em <- function(phy, X, params_init, edges, defaults,
       ll_total = ll_total,
       objective = m_step1_result$objective,
       sigma = params$sigma,
-      alpha = params$alpha,
+      alpha = if (defaults$model == 'EB') NA_real_ else params$alpha,
+      beta = if (defaults$model == 'EB') params$beta else NA_real_,
+      beta_H = if (defaults$model == 'EB') params$beta * max(defaults$parsed_alt_tree$leaf_dists) else NA_r
    eal_,
       tau = params$tau #,
    ))
       # Print progress
```

```
     if (verbose) {
-      cat(sprintf("\nIter %3d | LL_total = %10.2f | sigma = %.6f | alpha = %.6f || tau = %.6f\n", iter, ll_
     total, params$sigma, params$alpha,params$tau))
+      if (defaults$model == 'EB') {
+        cat(sprintf("\nIter %3d | LL_total = %10.2f | sigma = %.6f | beta = %.6f || tau = %.6f\n", iter, ll
     _total, params$sigma, params$beta, params$tau))
+      } else {
+        cat(sprintf("\nIter %3d | LL_total = %10.2f | sigma = %.6f | alpha = %.6f || tau = %.6f\n", iter, l
     l_total, params$sigma, params$alpha, params$tau))
+      }
       cat(paste0(state_names, ': ', round(params$theta, 3) ), '\n')
       flush.console()
     }
```

R/analysis_utils.R

Total: 24 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/analysis_utils.R b/R/analysis_utils.R
index 9f1dbae..0ff61ba 100644
--- a/R/analysis_utils.R
+++ b/R/analysis_utils.R
@@ -27,8 +27,8 @@ extract_parameters <- function(results_list){
         reg <- res$settings[[2]]
         unq_regs <- res$defaults$parsed_alt_tree$unique_regimes

-        if (reg == 'BM1'){
-            unq_regs <- unq_regs[-1] # strip off root if BM1.
+        if (reg %in% c('BM1', 'EB')){
+            unq_regs <- unq_regs[-1] # fixed-root single-theta models
         }
         tt <- unlist(res$paras$theta)
         names(tt) <- paste0('theta_', unq_regs)
@@ -62,7 +62,7 @@ annotate_history <- function(dataset1, datasetid){
                ungroup() %>%
         # delta_AIC is kept: SCOUT() selects on delta_AIC == 0 (main.R).
         # OU\\d+ (not OU1) so multi-regime truths like OU4 are labelled, and OU12 is not read as OU1.
-        mutate(truth = str_extract(gene_name, 'BM1|OUM|OU\\d+'))
+        mutate(truth = str_extract(gene_name, 'BM1|EB|OUM|OU\\d+'))

     return(dataset2)
 }
```

R/model_selection.R

Total: 75 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/model_selection.R b/R/model_selection.R
index 38ed020..10eafdf 100644
--- a/R/model_selection.R
+++ b/R/model_selection.R
@@ -12,10 +12,10 @@
 #
 # Procedure per gene:
 #   1. winner = argmin IC over all candidates (ic = 'AICc' or 'AIC', recomputed from ll/k/n).
-#   2. winner is BM1 or OU1 -> keep it.
-#   3. winner is multi-regime -> LRT winner vs OU1 (mvMORPH convention: larger model is the alt,
+#   2. winner is BM1, OU1 or EB -> keep it (EB is decided by IC only).
+#   3. winner is OU multi-regime -> LRT winner vs OU1 (mvMORPH convention: larger model is the alt,
 #      df = k difference, negative LR clipped to 0). p * n_multi < alpha -> keep winner;
-#      otherwise on_fail = 'fallback' -> IC-best of BM1/OU1, or 'ambiguous' -> labelled ambiguous.
+#      otherwise on_fail = 'fallback' -> IC-best simple model (BM1/OU1/EB), or 'ambiguous'.
 # lrt_status: 'not tested' (BM1 best), 'not eligible' (OU1 best; no valid LRT vs BM1),
 #             'tested: significant' / 'tested: not significant' (multi-regime winner vs OU1).
 # The statistics use SCOUT's unpenalized ll_total at penalized estimates: report as non-standard.
@@ -55,9 +55,12 @@ hybrid_decide <- function(reg, ll, k, n, a) {
   crit <- if (ic == "AIC") -2 * ll + 2 * k else -2 * ll + 2 * k * n / (n - k - 1)
   names(crit) <- names(ll) <- names(k) <- reg

-  n_multi <- sum(!reg %in% c(bm, ou))
+  # EB is a simple single-regime alternative, not an OU multi-regime LRT alt.
+  simple_reg <- c(bm, ou, 'EB')
+  n_multi <- sum(!reg %in% simple_reg)
   winner  <- reg[which.min(crit)]
-  simple  <- if (crit[bm] <= crit[ou]) bm else ou
+  simple_crit <- crit[intersect(simple_reg, reg)]
+  simple <- names(simple_crit)[which.min(simple_crit)]

   out <- list(
```

```
     ic               = ic,
@@ -89,6 +92,11 @@ hybrid_decide <- function(reg, ll, k, n, a) {
                               ou, ic, bm)
     return(out)
   }
+  if (winner == 'EB') {
+    out$lrt_status <- 'not eligible'
+    out$flag <- sprintf('EB best by %s; not an OU multi-regime alternative', ic)
+    return(out)
+  }

   raw  <- 2 * (ll[[winner]] - ll[[ou]])
   stat <- max(raw, 0)
@@ -364,6 +372,8 @@ scout_model_selection <- function(history,
   model_all <- as.character(df$model)
   gene_all  <- as.character(df$gene_name)
   alpha_all <- if ("alpha" %in% names(df)) df$alpha else NULL
+  beta_all  <- if ("beta" %in% names(df)) df$beta else NULL
+  betaH_all <- if ("beta_H" %in% names(df)) df$beta_H else NULL
   sigma_all <- if ("sigma" %in% names(df)) df$sigma else NULL
   tau_all   <- if ("tau" %in% names(df)) df$tau else NULL

@@ -380,7 +390,8 @@ scout_model_selection <- function(history,
         next_best_regime = NA_character_),
     stats::setNames(list(NA_real_, NA_real_, NA_real_), c(ic_col, w_col, d_col)),
     list(ll_total = ll_all[NA_integer_][1], param.count = k_all[NA_integer_][1],
-         ntips = n_all[NA_integer_][1], alpha = NA_real_, sigma = NA_real_, tau = NA_real_),
+         ntips = n_all[NA_integer_][1], alpha = NA_real_, beta = NA_real_,
+         beta_H = NA_real_, beta_near_bm = NA, sigma = NA_real_, tau = NA_real_),
     stats::setNames(rep(list(NA_real_), length(theta_cols)), theta_cols),
     list(iter = it_all[NA_integer_][1], converge = NA_character_,
         n_candidates_not_converged = NA_integer_),
@@ -427,6 +438,10 @@ scout_model_selection <- function(history,
     row$param.count <- k[i]
     row$ntips       <- n
     row$alpha <- if (mdl == "BM1" || is.null(alpha_all)) NA_real_ else alpha_all[j]
+    row$beta <- if (mdl != 'EB' || is.null(beta_all)) NA_real_ else beta_all[j]
+    row$beta_H <- if (mdl != 'EB' || is.null(betaH_all)) NA_real_ else betaH_all[j]
+    # A 1% maximum change of evolutionary rate across the tree is near BM.
+    row$beta_near_bm <- if (is.na(row$beta_H)) NA else abs(row$beta_H) <= 0.01
     row$sigma <- if (is.null(sigma_all)) NA_real_ else sigma_all[j]
     row$tau   <- if (is.null(tau_all)) NA_real_ else tau_all[j]
     if (length(theta_cols)) {
```

R/dropout_validation.R

Total: 51 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/dropout_validation.R b/R/dropout_validation.R
index fffbb94..d032b73 100644
--- a/R/dropout_validation.R
+++ b/R/dropout_validation.R
@@ -212,7 +212,7 @@ fit_paras <- function(res) {
     paras <- res$paras
     theta <- unlist(paras$theta)
     regs <- as.character(res$defaults$parsed_alt_tree$unique_regimes)
-    if (identical(res$defaults$model, 'BM1')) {
+    if (res$defaults$model %in% c('BM1', 'EB')) {
         regs <- regs[-1]
     }
     if (length(regs) != length(theta)) {
@@ -279,8 +279,10 @@ predict_masked_latent <- function(paras, defaults, held_idx, X_full, kept_idx =

     defaults <- align_regimes(defaults, paras$theta)

-    V <- compute_VCV(defaults$parsed_alt_tree, paras$alpha, paras$sigma, add.root = defaults$add.root)
-    W_ <- compute_W_matrix(defaults$parsed_alt_tree, paras$alpha, add.root = defaults$add.root)
+    V <- if (defaults$model == 'EB') compute_VCV_EB(defaults$parsed_alt_tree, paras$beta, paras$sigma) else
+        compute_VCV(defaults$parsed_alt_tree, paras$alpha, paras$sigma, add.root = defaults$add.root)
+    W_ <- if (defaults$model == 'EB') compute_W_EB(defaults$parsed_alt_tree) else
+        compute_W_matrix(defaults$parsed_alt_tree, paras$alpha, add.root = defaults$add.root)

     # BM1 with a fixed root carries theta twice, as in e_step.
     if (defaults$add.root == TRUE & defaults$model == 'BM1') {
@@ -694,7 +696,7 @@ runSCOUT.dropout <- function(tree,
             stop('`data` has duplicate species. formatSCOUT uses them as row names.')
         }
         if (is.null(states)) {
-            mcols <- if (is.null(mask_states)) setdiff(regimes, c('BM1', 'OU1')) else mask_states
+            mcols <- if (is.null(mask_states)) setdiff(regimes, c('BM1', 'OU1', 'EB')) else mask_states
```

```
            if (length(mcols) == 0) {
                stop('No regime columns to build mask states from. Set `mask_states` or `states`.')
            }
@@ -998,11 +1000,15 @@ runSCOUT.dropout <- function(tree,
                    param_rows[[length(param_rows) + 1]] <- data.frame(
                        replicate = r, arm = arm, gene = gene, regime = reg, true_model = ref,
                        alpha = res_m$paras$alpha %||% NA_real_,
+                       beta = res_m$paras$beta %||% NA_real_,
+                       beta_H = res_m$final$beta_H %||% NA_real_,
                        sigma = res_m$paras$sigma %||% NA_real_,
                        tau = res_m$paras$tau %||% NA_real_,
                        ll_total = aic_m$ll, param_count = aic_m$k, aic = aic_m$aic, aicc = aic_m$aicc,
                        converge = converge_m, n_kept = n_kept,
                        alpha_full = res_f$paras$alpha %||% NA_real_,
+                       beta_full = res_f$paras$beta %||% NA_real_,
+                       beta_H_full = res_f$final$beta_H %||% NA_real_,
                        sigma_full = res_f$paras$sigma %||% NA_real_,
                        tau_full = res_f$paras$tau %||% NA_real_,
                        stringsAsFactors = FALSE)
```

R/main.R

Total: 23 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/R/main.R b/R/main.R
index dce18bc..86e763c 100644
--- a/R/main.R
+++ b/R/main.R
@@ -2,7 +2,8 @@
 #' @param counts.file path to a counts file with a columns for species and relevent columns for evolutionar
     y models being tested if an OUX. Other columns will be genes.
 #' @param tree.file path to newick file. tips match species names in counts.
 #' @param results_dir output directory
-#' @param regimes a list of regimes to test. This include specifying BM1, OU1, and OUx where OUx correspond
      to columns in the counts file.
+#' @param regimes models to test: BM1, EB, OU1, and OUx partitions. EB uses
+#'   a fixed root and beta <= 0; OUx names must be columns of the counts file.
 #' @param species_key if the species column is not called 'species' (rather cellBC for example), indicate t
     he correct name here.
 #' @param blacklist genes for testing are inferred automatically. if any columns in the counts file are not
      for testing and not either the species key or model name, list them here.
 #' @param method one of EM (expectation-maximization), SM (smoothing), MTF (tip fog).
@@ -159,7 +160,7 @@ SCOUT <- function(counts.file, tree.file, results_dir,
   write.csv(annotated, sprintf('%s/%s_all_genes_full_history.csv', results_dir, tid))

   annotate_history_select <- annotated %>% filter(delta_AIC == 0) %>%
-    dplyr::select(iter, ll_total, sigma, alpha, tau, converge, gene_name, regime, model, param.count, ntips
     , AIC, AIC_weight)
+    dplyr::select(iter, ll_total, sigma, alpha, beta, beta_H, tau, converge, gene_name, regime, model, para
     m.count, ntips, AIC, AIC_weight)
    names(annotate_history_select)[which(names(annotate_history_select) == 'll_total')] <- 'loglik'
   write.csv(annotate_history_select, sprintf('%s/%s_annotated_best_fit.csv', results_dir, tid))
```

README.md

Total: 12 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/README.md b/README.md
index c16ab07..985d571 100644
--- a/README.md
+++ b/README.md
@@ -126,6 +126,7 @@ This returns a list with a dataframe of gene annotations (best fit models) and a
 ### Understanding the Models

 - **BM1** (Brownian Motion): Models trait evolution as a random walk with constant variance. Assumes no sel
     ection.
+- **EB** (Early Burst): Models a variance rate `sigma * exp(beta * t)`, with `t` measured from the root and
      `beta <= 0`. The ancestral mean is estimated with a fixed root (as for BM1), even when `fixed_root = F
     ALSE`; positive `beta` would be AC, not EB. Use `regimes = c("BM1", "EB", "OU1", "OUM")` without adding
      an `EB` column to the counts. `beta_H = beta * H` reports the dimensionless rate change across the tre
     e; `beta_near_bm` flags `abs(beta_H) <= 0.01` without changing model selection. `scale_tree = FALSE` ke
     eps `beta` in the original branch-length units.
 - **OU1** (Single Ornstein-Uhlenbeck): Models evolution toward a single optimum with selection strength α a
     nd trait optimum θ.
 - **OUM** (Multiple Optima OU): Models evolution with different trait optima for different regimes (e.g., d
     ifferent cell types or lineages).
```

man/SCOUT.Rd

Total: 14 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/man/SCOUT.Rd b/man/SCOUT.Rd
index e786687..7a2e41d 100644
--- a/man/SCOUT.Rd
+++ b/man/SCOUT.Rd
@@ -42,7 +42,8 @@ SCOUT(

 \item{results_dir}{output directory}

-\item{regimes}{a list of regimes to test. This include specifying BM1, OU1, and OUx where OUx correspond to
      columns in the counts file.}
+\item{regimes}{models to test: BM1, EB, OU1, and OUx partitions. EB uses
+a fixed root and beta <= 0; OUx names must be columns of the counts file.}

 \item{species_key}{if the species column is not called 'species' (rather cellBC for example), indicate the
     correct name here.}
```

tests/testthat/test-early-burst.R

Total: 134 líneas del diff; se incluyen todos los bloques de cambio.

```
diff --git a/tests/testthat/test-early-burst.R b/tests/testthat/test-early-burst.R
new file mode 100644
--- /dev/null
+++ b/tests/testthat/test-early-burst.R
@@ -0,0 +1,129 @@
+test_that('EB covariance matches transformed tree and tends to BM at beta = 0', {
+    for (nwk in c('((a:1,b:1):1,(c:1,d:1):1);',
+                  '((a:0.5,b:1):1,(c:1,d:2):0.5);')) {
+        phy <- ape::read.tree(text = nwk)
+        depths <- ape::node.depth.edgelength(phy)
+        info <- list(n_leaves = 4L, leaf_dists = depths[1:4],
+                     shared_lengths = matrix(depths[ape::mrca(phy)], 4, 4))
+        for (beta_H in c(-5, -1, -0.01, -1e-9, 0)) {
+            beta <- beta_H / max(info$leaf_dists)
+            actual <- SCOUT:::compute_VCV_EB(info, beta, 1.3)
+            tr <- SCOUT:::transformPhy_EB(phy, beta, 1.3)
+            expect_equal(actual, unname(ape::vcv(tr$tree)), tolerance = 1e-8)
+            expect_equal(tr$tree$root.edge, 0)
+            expect_equal(unname(tr$diagWeight), rep(1, 4))
+        }
+        expect_equal(SCOUT:::compute_VCV_EB(info, 0, 1.3),
+                     unname(1.3 * ape::vcv(phy)))
+        expect_error(SCOUT:::compute_VCV_EB(info, 0.1, 1.3), 'beta <= 0', fixed = TRUE)
+        expect_error(SCOUT:::transformPhy_EB(phy, 0.1, 1.3), 'beta <= 0', fixed = TRUE)
+        scaled <- info
+        scaled$leaf_dists <- 100 * info$leaf_dists
+        scaled$shared_lengths <- 100 * info$shared_lengths
+        expect_equal(SCOUT:::compute_VCV_EB(info, -0.3, 1.3),
+                     SCOUT:::compute_VCV_EB(scaled, -0.3 / 100, 1.3 / 100))
+    }
+})
+
+test_that('EB is a single-regime model with fixed root regardless of user default', {
+    phy <- ape::read.tree(text = '((a:1,b:1):1,(c:1,d:1):1);')
+    meta <- data.frame(species = phy$tip.label, gene = c(2, 3, 4, 5))
+    idata <- SCOUT::formatSCOUT(phy, meta, tempdir(), regimes = c('BM1', 'OU1', 'EB'),
+                                anc_infer = 'skip', normalize = FALSE)
+    expect_equal(idata$gene_cols, 'gene')
+    for (fixed in c(FALSE, TRUE)) {
+        ti <- SCOUT:::preprocessTree(idata, 'EB', root.fixed = fixed)
+        expect_true(ti$defaults$add.root)
+        expect_equal(ti$defaults$model, 'EB')
+        gi <- SCOUT:::preprocessGene(idata, 'gene', ti$tree_const, ti$defaults$root.state,
+                                     ti$defaults$add.root, 'EB', FALSE)
+        expect_lte(gi$param_init$beta, 0)
+        expect_null(gi$param_init$alpha)
+        expect_length(gi$param_init$theta, 1)
+    }
+})
+
+test_that('EB near-BM flag uses beta times tree height without forcing a BM call', {
+    rows <- data.frame(gene_name = 'g', regime = c('BM1', 'OU1', 'EB'),
+                       model = c('BM1', 'OU1', 'EB'), ll_total = c(-100, -100, -95),
+                       param.count = c(3, 4, 4), ntips = 100,
+                       beta = c(NA, NA, -0.0001), beta_H = c(NA, NA, -0.01),
+                       converge = 'converged', iter = 8L)
+    res <- SCOUT::scout_model_selection(rows, hybrid = TRUE)
+    expect_equal(res$selected_regime, 'EB')
```

```
+    expect_true(res$beta_near_bm)
+    expect_equal(res$lrt_status, 'not eligible')
+    expect_equal(res$beta_H, -0.01)
+    rows$beta_H[3] <- -0.011
+    expect_false(SCOUT::scout_model_selection(rows, hybrid = TRUE)$beta_near_bm)
+})
+
+test_that('hybrid LRT excludes EB from OU multi-regime tests and fallback', {
+    rows <- data.frame(regime = c('BM1', 'OU1', 'EB', 'OU2'),
+                       ll_total = c(-120, -110, -109.9, -108.8),
+                       param.count = c(3, 4, 4, 5), ntips = 100)
+    res <- SCOUT::select_model_hybrid(rows, ic = 'AIC')
+    expect_equal(res$ic_winner, 'OU2')
+    expect_equal(res$n_multi, 1L)
+    expect_equal(res$best_simple, 'EB')
+    expect_equal(res$call, 'EB')
+    expect_equal(res$lrt_alt, 'OU2')
+    expect_equal(res$lrt_null, 'OU1')
+})
+
+test_that('EB M-step and E-step use the same covariance and count beta', {
+    phy <- ape::read.tree(text = '((a:1,b:1):1,(c:1,d:1):1);')
+    meta <- data.frame(species = phy$tip.label, gene = c(2, 3, 4, 5))
+    idata <- SCOUT::formatSCOUT(phy, meta, tempdir(), regimes = 'EB',
+                                anc_infer = 'skip', normalize = FALSE)
+    ti <- SCOUT:::preprocessTree(idata, 'EB', root.fixed = FALSE)
+    gi <- SCOUT:::preprocessGene(idata, 'gene', ti$tree_const, ti$defaults$root.state,
+                                ti$defaults$add.root, 'EB', FALSE)
+    d <- ti$defaults
+    d$verbose <- FALSE
+    d$tau_prior_mean <- 0.2
+    d$tau_prior_sd <- 0.1
+    er <- SCOUT:::e_step(gi$X, ti$tree_const$tree, ti$tree_const$edges,
+                         gi$param_init, d, diagnose = FALSE)
+    expect_length(er$E_Z, 4)
+    pred <- SCOUT:::predict_masked_latent(gi$param_init, d, seq_len(4),
+                                          gi$X, kept_idx = seq_len(4))
+    expect_equal(unname(as.vector(er$E_Z)), unname(pred$mean), tolerance = 1e-8)
+    fit <- SCOUT:::m_step_tipfog(gi$X, gi$X, gi$param_init,
+                                 ti$tree_const$tree, ti$tree_const$edges, d, 0.2, 0.2)
+    expect_lte(fit$beta, 0)
+    V <- SCOUT:::compute_VCV_EB(d$parsed_alt_tree, fit$beta, fit$sigma)
+    diag(V) <- diag(V) + fit$tau^2
+    expect_equal(fit$ll_total,
+                 as.numeric(-0.5 * (4 * log(2 * pi) + as.numeric(determinant(V, logarithm = TRUE)$modulus)
    +
+                                    t(gi$X - fit$theta) %*% solve(V, gi$X - fit$theta))),
+                 tolerance = 1e-7)
+    out <- SCOUT:::run_em(ti$tree_const$tree, gi$X, gi$param_init,
+                          ti$tree_const$edges, d, max_iter = 1, skipE = TRUE,
+                          save = FALSE, diagnose = FALSE, verbose = FALSE)
+    expect_equal(length(unlist(out$paras)), 4)
+    expect_equal(out$final$beta_H,
+                 out$paras$beta * max(d$parsed_alt_tree$leaf_dists))
+})
+
+test_that('EB passes through runSCOUT parameter extraction and IC accounting', {
+    phy <- ape::read.tree(text = '((a:1,b:1):1,(c:1,d:1):1);')
+    meta <- data.frame(species = phy$tip.label, gene = c(2, 3, 4, 5),
+                       gene2 = c(3, 2, 5, 4))
+    idata <- SCOUT::formatSCOUT(phy, meta, tempdir(), regimes = 'EB',
+                                anc_infer = 'skip', normalize = FALSE)
+    fits <- SCOUT::runSCOUT(idata, lambda1 = 0.2, lambda2 = 0.2,
+                            fixed.root = FALSE, M_only = TRUE, cores = 1, verbose = FALSE)
+    hist <- SCOUT:::extract_history_grid_search(fits)
+    expect_true(all(hist$param.count == 4))
+    expect_true(all(hist$model == 'EB'))
+    pars <- SCOUT::extract_parameters(fits)
+    expect_equal(length(pars$EB$gene$theta_1), 1)
+    expect_true(is.finite(pars$EB$gene$beta))
+    expect_equal(hist$beta_H,
+                 hist$beta * max(fits[[1]]$defaults$parsed_alt_tree$leaf_dists))
+    idata$gene_cols <- 'gene'
+    single <- SCOUT::runSCOUT(idata, lambda1 = 0.2, lambda2 = 0.2,
+                              fixed.root = FALSE, M_only = TRUE, cores = 1, verbose = FALSE)
+    expect_length(single, 1)
+})
```

Archivos binarios y alcance

docs/SCOUT_EB_resumen.pdf fue añadido anteriormente como resumen general; este nuevo PDF contiene el detalle verificable del código. No se incluyen archivos no modificados del repositorio. El diff muestra el estado actual, no una versión instalada en RStudio ni cambios no guardados allí.